Project Summary

1. Project Purpose and Main Features

Purpose

The project is an automated code analysis and documentation generation CLI tool. By leveraging the Google GenAI SDK (Gemini models), the application recursively scans local source code repositories, performs automated static code reviews at both the file and project levels, and exports a comprehensive summary report as a formatted PDF.

Main Features

- **Multi-Language File Discovery**: Automatically scans project directories and filters relevant source code, configuration, and documentation files across 60+ supported extensions (`config.py`).
- **Automated File-Level Analysis**: Analyzes individual source files using Gemini models (`gemini-3.5-flash-lite`) and standardized prompt templates (`FILE_ANALYSIS_PROMPT`) to identify purpose, key components, and potential code improvements.
- **Aggregated Project Summarization**: Aggregates all file-level reviews to generate an overarching project summary (`PROJECT_SUMMARY_PROMPT`) via Gemini models (`gemini-3.6-flash`).
- **PDF Report Generation**: Converts Markdown summary reports into HTML and compiles them into downloadable PDF documents using `xhtml2pdf`.
- **Basic Rate-Limit Mitigation**: Incorporates synchronous execution delays (`time.sleep(3)`) between API requests to prevent hitting Gemini API quotas during file reviews.

---

2. Architecture and Relationships Between Components

The application follows a simple modular architecture that separates CLI entry logic, domain operations, configuration data, and prompt engineering.

- **`main.py` (CLI Entry Point & Orchestrator)**
- Receives user arguments (repository path) via `argparse`.
- Instantiates the Google GenAI client (`genai.Client`) using the `GEMINI_API_KEY` environment variable.

  Creates an instance of `Repository` and coordinates the execution flow:

  - Invokes `scan()` to identify files.
  - Invokes `analyze_files()` to generate file-level summaries.
  - Invokes `summarize_project()` to synthesize overall codebase structure.
  - Invokes `export_pdf()` to write the output report.

  `repository.py` (Core Engine / `Repository` Class)

- Manages file system interactions and AI client interactions.

  Dependencies:

  - Reads extension constraints from `config.py` (`SUPPORTED_EXTENSIONS`).
  - Imports prompt templates from `prompts.py` (`FILE_ANALYSIS_PROMPT` and `PROJECT_SUMMARY_PROMPT`).
  - Communicates with `genai.Client` to request LLM responses.
  - Uses `xhtml2pdf` to transform Markdown/HTML outputs into PDF format.

  `config.py` (Configuration Layer)

- Defines the global `SUPPORTED_EXTENSIONS` set.

  Used directly by `Repository.scan()` to filter valid files during directory traversal.

  `prompts.py` (Prompt Engineering Layer)

- Defines multiline prompt string templates:
  - `FILE_ANALYSIS_PROMPT`: Directs the AI on how to evaluate individual files.
  - `PROJECT_SUMMARY_PROMPT`: Directs the AI on how to synthesize all file summaries into a global project overview.
- Extracted by `Repository` methods (`analyze_files` and `summarize_project`) before making API calls.

---

3. Overall Code Quality and Improvement Suggestions

Overall Code Quality

The codebase exhibits a clean separation of concerns, keeping configuration, prompt strings, domain logic, and CLI entry points in distinct modules. The workflow is straight-forward and easy to follow. However, the current implementation lacks robust error handling, flexibility, and performance optimizations suitable for production use or handling larger repositories.

Potential Improvement Suggestions

1. **Error Handling & Resilience**
2. **API and Path Validation**: Add explicit checks for `GEMINI_API_KEY` presence in `main.py` and validate that the user-provided repository directory exists before proceeding.
3. **API Fault Tolerance**: Wrap GenAI API calls in `analyze_files()` and `summarize_project()` with retry logic and `try-except` blocks to handle network timeouts, rate-limit errors, or invalid responses gracefully.

   File Processing Failures: Enhance PDF generation in `export_pdf()` to gracefully catch conversion errors.

   Performance Optimization

   Asynchronous Processing: Replace the blocking `time.sleep(3)` loop with asynchronous API calls (`asyncio`) or concurrent worker pools, paired with dynamic rate-limit throttling (exponential backoff).

   Configuration & Flexibility

8. **Externalize Settings**: Move hardcoded parameters—such as Gemini model names (`gemini-3.5-flash-lite`, `gemini-3.6-flash`), sleep durations, and default export filenames (`summary.pdf` / `project_summary.pdf`)—to CLI arguments or an external configuration file (e.g., YAML/JSON).

   Case-Insensitive File Matching: Update `config.py` extension filtering to normalize file paths to lowercase (e.g., handle `.PY` or `.CPP`). Group extensions by category (e.g., language family) for better code readability.

   Prompt Management

11. **Template Engines**: Transition from hardcoded multiline Python strings in `prompts.py` to structured template engine files (such as Jinja2 or `string.Template`) or external YAML configurations to allow users to customize analysis guidelines without altering code.